

AlarmValidatorTest

```
import ec.edu.espe.mothersApp.model.AlarmValidator;

import org.junit.Before;
import org.junit.Test;
import static org.junit.Assert.*;
/**
 *
 * @author Cristian
 */
public class AlarmValidatorTest {
    private AlarmValidator validator;

    @Before
    public void setUp() {
        validator = new AlarmValidator();
    }

    @Test
    public void testTemperatureNormal() {
        assertTrue(validator.checkTemperature(36.5));
    }

    @Test
    public void testTemperatureFever() {
        assertFalse(validator.checkTemperature(38.5));
    }

    @Test
    public void testTemperatureLowerBoundary() {
        assertTrue(validator.checkTemperature(35.5));
    }

    @Test
    public void testTemperatureUpperBoundary() {
        assertTrue(validator.checkTemperature(38.0));
    }

    @Test
    public void testTemperatureHypothermia() {
        assertFalse(validator.checkTemperature(35.0));
    }

    @Test
    public void testRespirationNormal() {
        assertTrue(validator.checkRespiration("no"));
    }

    @Test
    public void testRespirationDistress() {
        assertFalse(validator.checkRespiration("yes"));
    }
}
```

```
}

@Test
public void testRespirationUpperCase() {
    assertFalse validator.checkRespiration("YES");
}

@Test
public void testFeedingNormal() {
    assertTrue validator.checkFeeding("no");
}

@Test
public void testFeedingProblem() {
    assertFalse validator.checkFeeding("yes");
}

@Test
public void testSkinColorNormal() {
    assertTrue validator.checkSkinColor("no");
}

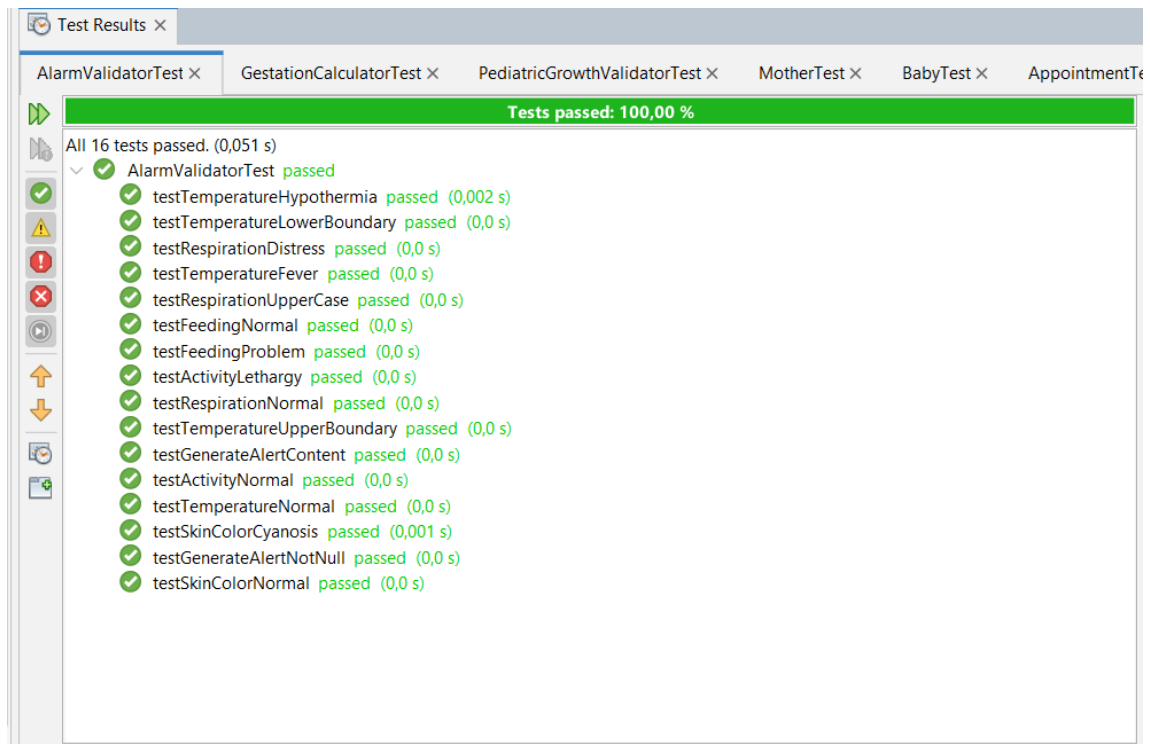
@Test
public void testSkinColorCyanosis() {
    assertFalse validator.checkSkinColor("yes");
}

@Test
public void testActivityNormal() {
    assertTrue validator.checkActivity("no");
}

@Test
public void testActivityLethargy() {
    assertFalse validator.checkActivity("yes");
}

@Test
public void testGenerateAlertNotNull() {
    assertNotNull validator.generateAlert();
}

@Test
public void testGenerateAlertContent() {
    String alert = validator.generateAlert();
    assertTrue(alert.toUpperCase().contains("CRITICAL"));
}
}
```



GestationCalculatorTest

```
import ec.edu.espe.mothersApp.model.GestationCalculator;

import org.junit.Test;
import static org.junit.Assert.*;

/**
 *
 * @author Cristian
 */
public class GestationCalculatorTest {
    @Test
    public void testWeek6Is1stTrimester() {
        GestationCalculator.GestationResult r =
            GestationCalculator.calculateFromWeek(6);
        assertEquals("1st Trimester", r.trimester);
        assertEquals(6, r.currentWeek);
    }

    @Test
    public void testWeek12Is1stTrimester() {
        GestationCalculator.GestationResult r =
            GestationCalculator.calculateFromWeek(12);
        assertEquals("1st Trimester", r.trimester,
            "Week 12 is the last week of 1st trimester");
    }

    @Test
```

```

public void testWeek13Is2ndTrimester() {
    GestationCalculator.GestationResult r =
        GestationCalculator.calculateFromWeek(13);
    assertEquals("2nd Trimester", r.trimester);
}

@Test
public void testWeek27Is2ndTrimester() {
    GestationCalculator.GestationResult r =
        GestationCalculator.calculateFromWeek(27);
    assertEquals("2nd Trimester", r.trimester,
        "Week 27 is the last week of 2nd trimester");
}

@Test
public void testWeek28Is3rdTrimester() {
    GestationCalculator.GestationResult r =
        GestationCalculator.calculateFromWeek(28);
    assertEquals("3rd Trimester", r.trimester);
}

@Test
public void testWeek40Is3rdTrimester() {
    GestationCalculator.GestationResult r =
        GestationCalculator.calculateFromWeek(40);
    assertEquals("3rd Trimester", r.trimester,
        "Week 40 is full term");
}

@Test
public void testMethodFieldIsWeek() {
    GestationCalculator.GestationResult r =
        GestationCalculator.calculateFromWeek(20);
    assertEquals("WEEK", r.method);
}

@Test
public void testDueDateNotNull() {
    GestationCalculator.GestationResult r =
        GestationCalculator.calculateFromWeek(20);
    assertNotNull(r.estimatedDueDate,
        "Estimated due date must never be null");
}

@Test
public void testLmpMethodField() {
    GestationCalculator.GestationResult r =
        GestationCalculator.calculateFromLMP("2025-01-01");
    assertEquals("LMP", r.method);
}

@Test

```

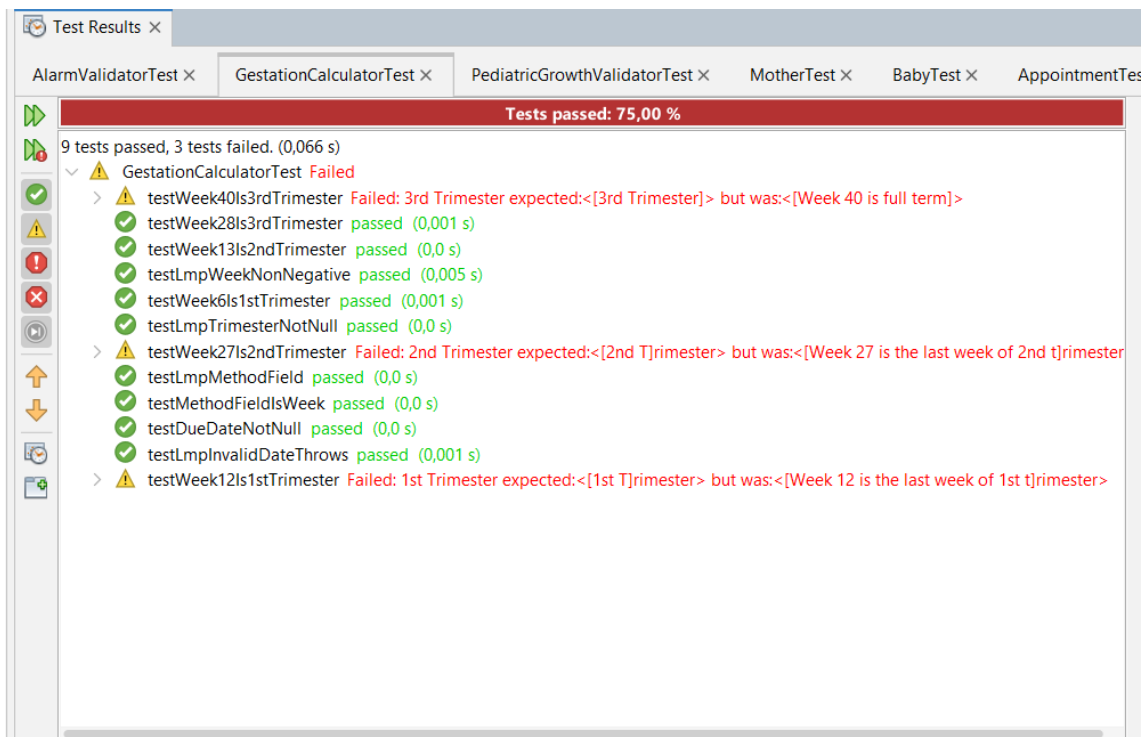
```

public void testLmpWeekNonNegative() {
    GestationCalculator.GestationResult r =
        GestationCalculator.calculateFromLMP("2025-01-01");
    assertTrue("Gestation week cannot be negative",
        r.currentWeek >= 0);
}

@Test
public void testLmpTrimesterNotNull() {
    GestationCalculator.GestationResult r =
        GestationCalculator.calculateFromLMP("2025-06-01");
    assertNotNull("Trimester must always be assigned",
        r.trimester);
}

@Test
public void testLmpInvalidDateThrows() {
    assertThrows("Invalid date format should throw an exception",
        Exception.class,
        () -> GestationCalculator.calculateFromLMP("not-a-date"));
}
}

```



PediatricGrowthValidatorTest

```

import ec.edu.espe.mothersApp.model.PediatricGrowthValidator;

import java.io.ByteArrayOutputStream;

```

```

import java.io.PrintStream;
import org.junit.Test;
import static org.junit.Assert.*;

/**
 *
 * @author Cristian
 */
public class PediatricGrowthValidatorTest {
    private String captureOutput(Runnable action) {
        ByteArrayOutputStream baos = new ByteArrayOutputStream();
        PrintStream original = System.out;
        System.setOut(new PrintStream(baos));
        action.run();
        System.setOut(original);
        return baos.toString();
    }

    @Test
    public void testNormalGrowth() {
        String output = captureOutput(() ->
            PediatricGrowthValidator.showResult(3200, 50));
        assertTrue("Full-term healthy baby should be NORMAL",
output.contains("NORMAL"));
    }

    @Test
    public void testNormalGrowthBoundary() {
        String output = captureOutput(() ->
            PediatricGrowthValidator.showResult(2500, 45));
        assertTrue("Exact minimum for NORMAL should still be NORMAL",
output.contains("NORMAL"));
    }

    @Test
    public void testLowRiskGrowth() {
        String output = captureOutput(() ->
            PediatricGrowthValidator.showResult(2000, 38));
        assertTrue("Underweight but above severe threshold should be LOW RISK",
output.contains("LOW RISK"));
    }

    @Test
    public void testLowRiskGrowthBoundary() {
        String output = captureOutput(() ->
            PediatricGrowthValidator.showResult(1800, 35));
        assertTrue("Exact minimum for LOW RISK should be LOW RISK",
output.contains("LOW RISK"));
    }

    @Test
    public void testJustBelowNormal() {

```

```
String output = captureOutput() ->
    PediatricGrowthValidator.showResult(2499, 44));
    assertTrue("Just below NORMAL threshold should be LOW RISK",
output.contains("LOW RISK"));
}

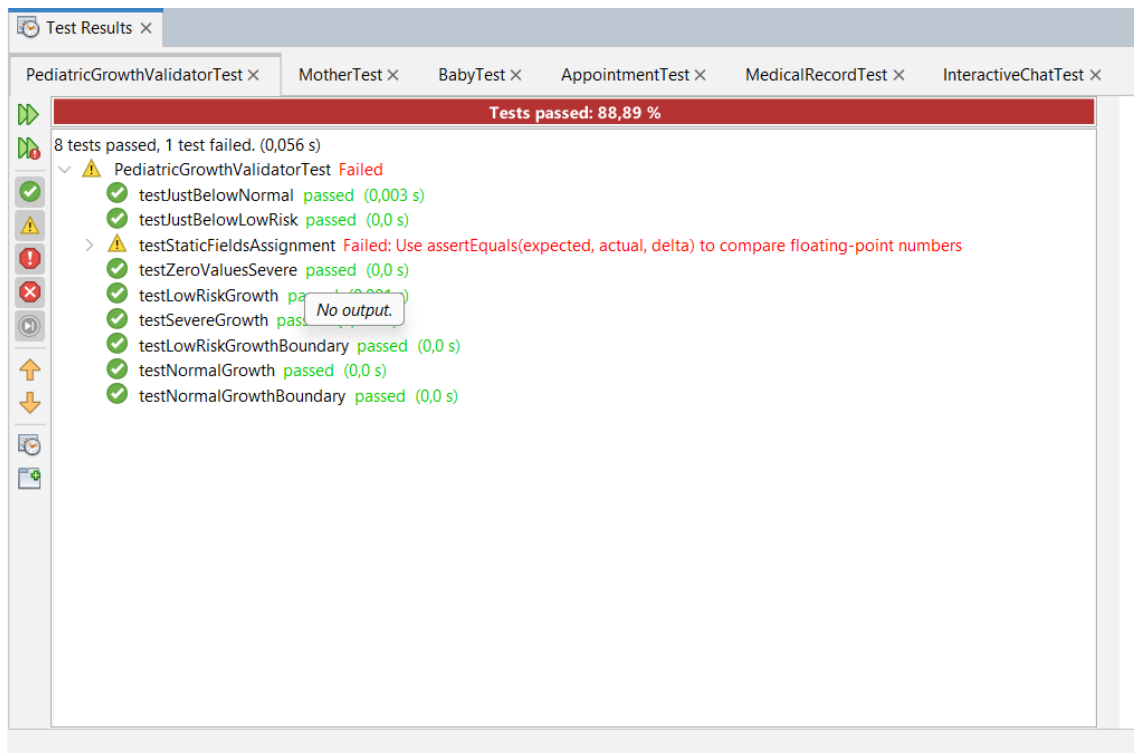
@Test
public void testSevereGrowth() {
    String output = captureOutput() ->
        PediatricGrowthValidator.showResult(1200, 30));
    assertTrue("Very underweight baby should be SEVERE",
output.contains("SEVERE"));
}

@Test
public void testZeroValuesSevere() {
    String output = captureOutput() ->
        PediatricGrowthValidator.showResult(0, 0));
    assertTrue("Zero values should always be SEVERE", output.contains("SEVERE"));
}

@Test
public void testJustBelowLowRisk() {
    String output = captureOutput() ->
        PediatricGrowthValidator.showResult(1799, 34));
    assertTrue("Just below LOW RISK threshold should be SEVERE",
output.contains("SEVERE"));
}

@Test
public void testStaticFieldsAssignment() {
    PediatricGrowthValidator.babyWeight = 3000;
    PediatricGrowthValidator.babyHeight = 50;
    PediatricGrowthValidator.motherWeight = 65;
    PediatricGrowthValidator.motherHeight = 162;

    assertEquals(3000, PediatricGrowthValidator.babyWeight);
    assertEquals(50, PediatricGrowthValidator.babyHeight);
}
}
```



MotherTest

```
import ec.edu.espe.mothersApp.model.Mother;

import org.junit.Before;
import org.junit.Test;
import static org.junit.Assert.*;

/**
 *
 * @author Cristian
 */
public class MotherTest {
    private Mother mother;

    @Before
    public void setUp() {
        mother = new Mother("Maria", "Gomez", "1712345678",
            "1990-05-15", 65.0, 162.0);
    }

    @Test
    public void testFirstName() {
        assertEquals("Maria", mother.firstName);
    }

    @Test
    public void testLastName() {
        assertEquals("Gomez", mother.lastName);
    }
}
```



```
}

@Test
public void testIdNumber() {
    assertEquals("1712345678", mother.getIdNumber());
}

@Test
public void testWeight() {
    assertEquals(65.0, mother.weight, 0.001);
}

@Test
public void testHeight() {
    assertEquals(162.0, mother.height, 0.001);
}

@Test
public void testDisabilityDefault() {
    assertFalse("Disability should default to false", mother.isDisability());
}

@Test
public void testBabiesListEmpty() {
    assertNotNull(mother.babies);
    assertEquals(0, mother.babies.size());
}

@Test
public void testValidateValidId() {
    assertTrue("Valid numeric ID should pass validation", mother.validate());
}

@Test
public void testValidateInvalidIdLetters() {
    mother.setIdNumber("ABCD123456");
    assertFalse("ID with letters should fail validation", mother.validate());
}

@Test
public void testValidateNullId() {
    mother.setIdNumber(null);
    assertFalse("Null ID should fail validation", mother.validate());
}

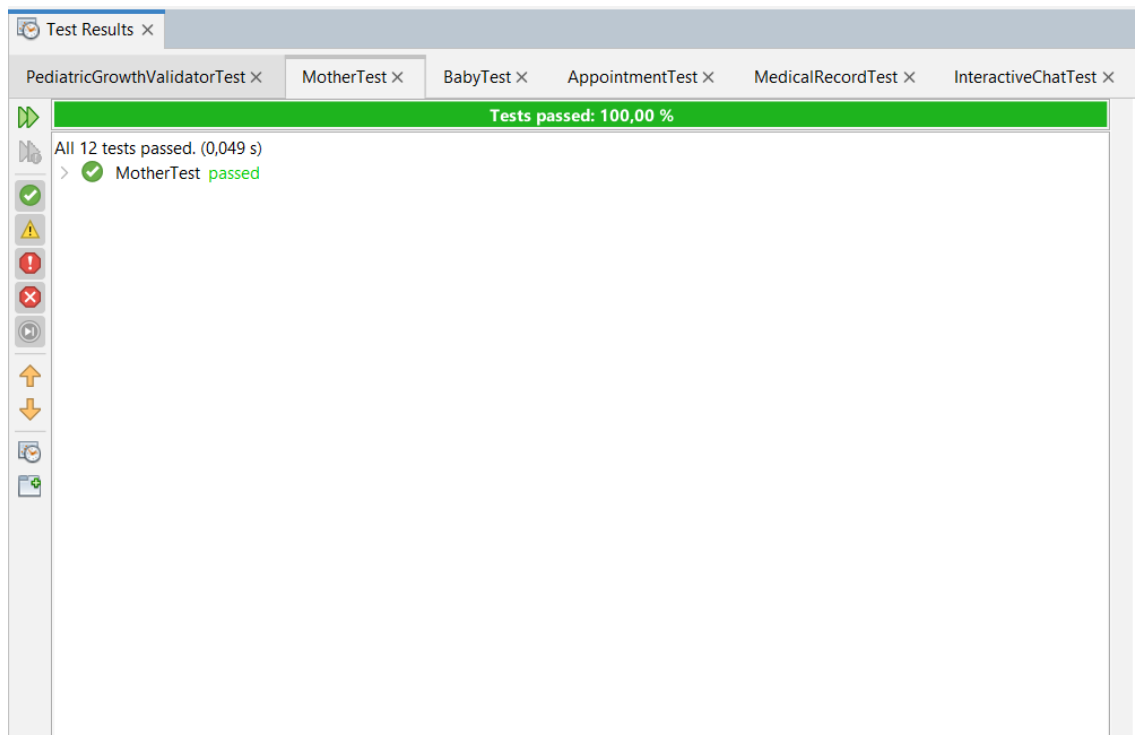
@Test
public void testSetAge() {
    mother.setAge(30);
    assertEquals(30, mother.getAge());
}

@Test
```

```

public void testSetDisability() {
    mother.setDisability(true);
    assertTrue(mother.isDisability());
}
}

```



BabyTest

```

import ec.edu.espe.mothersApp.model.Baby;

import org.junit.Before;
import org.junit.Test;
import static org.junit.Assert.*;

/**
 *
 * @author Cristian
 */
public class BabyTest {
    private Baby baby;

    @Before
    public void setUp() {
        baby = new Baby("Sofia", "Gomez", "B-001",
            3200.0, 50.0, "2024-01-10", true);
    }

    @Test

```

```
public void testFirstName() {
    assertEquals("Sofia", baby.firstName);
}

@Test
public void testWeight() {
    assertEquals(3200.0, baby.getWeight(), 0.001);
}

@Test
public void testHeight() {
    assertEquals(50.0, baby.getHeight(), 0.001);
}

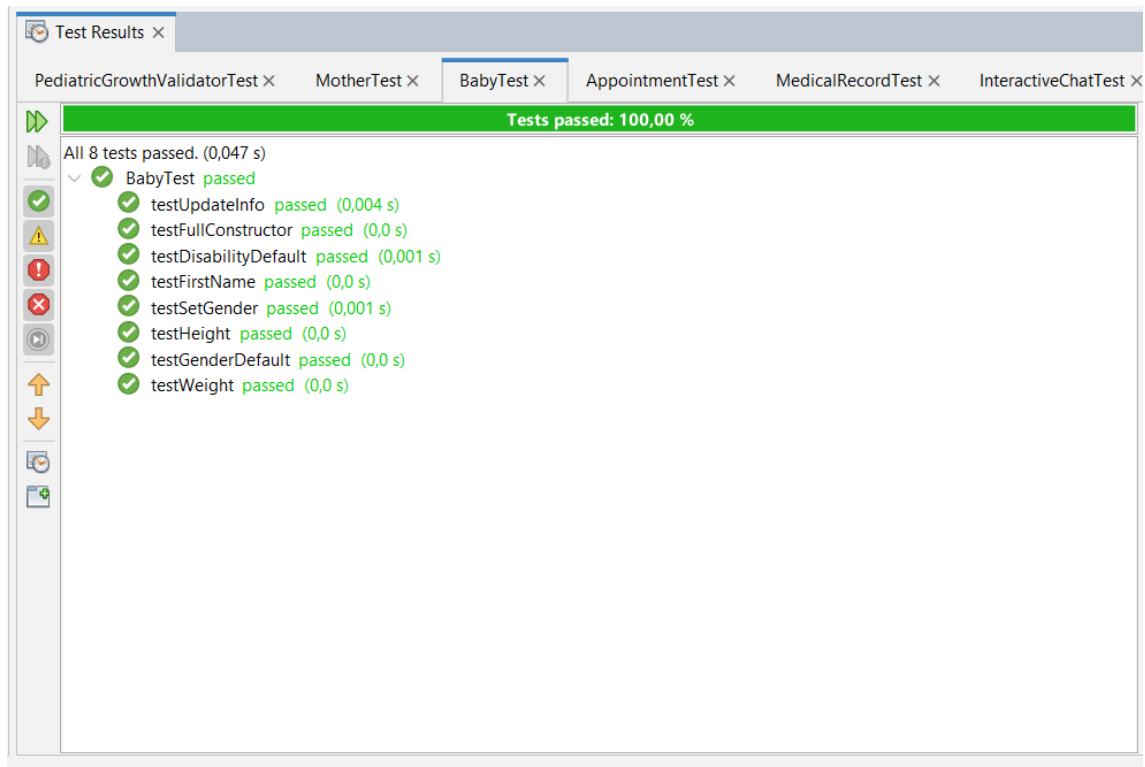
@Test
public void testGenderDefault() {
    assertEquals("", baby.getGender());
}

@Test
public void testDisabilityDefault() {
    assertFalse(baby.isDisability());
}

@Test
public void testUpdateInfo() {
    baby.updateInfo(3500.0, 52.0, false);
    assertEquals(3500.0, baby.getWeight(), 0.001);
    assertEquals(52.0, baby.getHeight(), 0.001);
    assertFalse(baby.isDisability());
}

@Test
public void testSetGender() {
    baby.setGender("Female");
    assertEquals("Female", baby.getGender());
}

@Test
public void testFullConstructor() {
    Baby fullBaby = new Baby("Lucas", "Torres", "B-002",
        2900.0, 48.0, "Male",
        "2024-03-01", true, false);
    assertEquals("Male", fullBaby.getGender());
    assertEquals(2900.0, fullBaby.getWeight(), 0.001);
    assertFalse(fullBaby.isDisability());
}
}
```



AppointmentTest

```
import ec.edu.espe.mothersApp.model.Appointment;

import org.junit.Before;
import org.junit.Test;
import java.sql.Time;
import java.util.Date;
import static org.junit.Assert.*;

/**
 *
 * @author Cristian
 */
public class AppointmentTest {
    private Appointment appointment;

    @Before
    public void setUp() {
        appointment = new Appointment();
        appointment.setMotherId(1);
        appointment.setDoctorId(2);
    }

    @Test
    public void testScheduleStatus() {
        Date date = new Date();
        Time time = new Time(System.currentTimeMillis());
```

```
        appointment.schedule(date, time, "Checkup");
        assertEquals("Scheduled", appointment.getStatus());
    }

    @Test
    public void testScheduleDate() {
        Date date = new Date();
        appointment.schedule(date, new Time(0), "Follow-up");
        assertEquals(date, appointment.getDate());
    }

    @Test
    public void testScheduleReason() {
        appointment.schedule(new Date(), new Time(0), "Vaccination");
        assertEquals("Vaccination", appointment.getReason());
    }

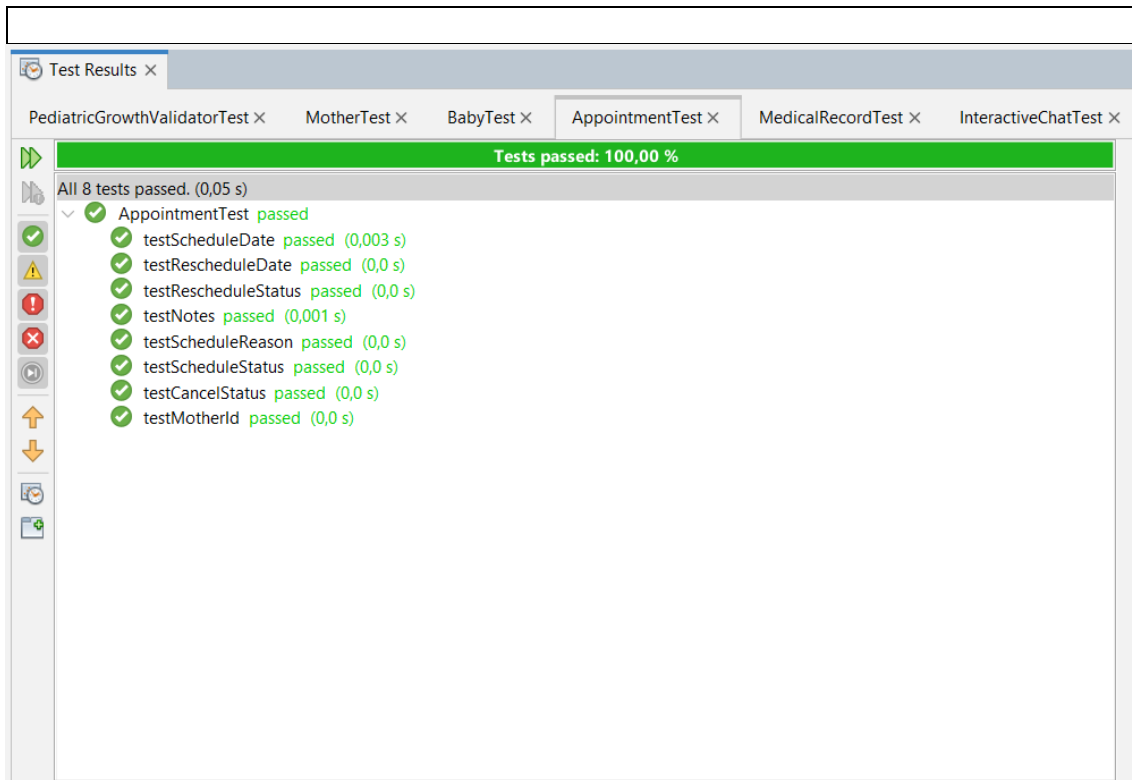
    @Test
    public void testCancelStatus() {
        appointment.cancel("Mother unavailable");
        assertEquals("Cancelled", appointment.getStatus());
    }

    @Test
    public void testRescheduleStatus() {
        Date newDate = new Date();
        appointment.reschedule(newDate, new Time(0));
        assertEquals("Rescheduled", appointment.getStatus());
    }

    @Test
    public void testRescheduleDate() {
        Date newDate = new Date(System.currentTimeMillis() + 86400000L);
        appointment.reschedule(newDate, new Time(0));
        assertEquals(newDate, appointment.getDate());
    }

    @Test
    public void testNotes() {
        appointment.setNotes("Bring medical documents");
        assertEquals("Bring medical documents", appointment.getNotes());
    }

    @Test
    public void testMotherId() {
        assertEquals(1, appointment.getMotherId());
    }
}
```



MedicalRecordTest

```
import ec.edu.espe.mothersApp.model.MedicalRecord;

import org.junit.Before;
import org.junit.Test;
import static org.junit.Assert.*;
/**
 *
 * @author Cristian
 */
public class MedicalRecordTest {
    private MedicalRecord record;

    @Before
    public void setUp() {
        record = new MedicalRecord(1, 70.0, 165.0, 36.8,
            "Normal", "Normal", "Normal", "Normal",
            "Routine checkup");
    }

    @Test
    public void testId() {
        assertEquals(1, record.getId());
    }

    @Test
    public void testTemperature() {
```

```
        assertEquals(36.8, record.getTemperature(), 0.001);
    }

    @Test
    public void testNotes() {
        assertEquals("Routine checkup", record.getNotes());
    }

    @Test
    public void testDefaultRespiratoryStatus() {
        MedicalRecord def = new MedicalRecord();
        assertEquals("Normal", def.getRespiratoryStatus());
    }

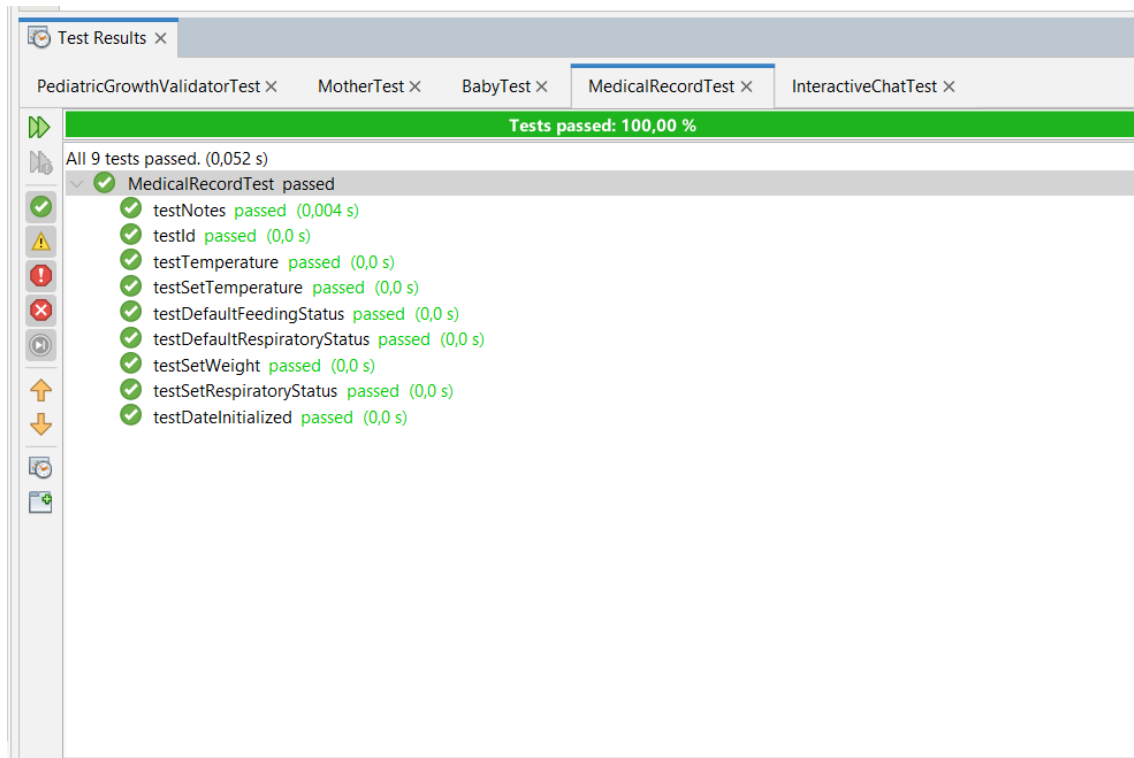
    @Test
    public void testDefaultFeedingStatus() {
        MedicalRecord def = new MedicalRecord();
        assertEquals("Normal", def.getFeedingStatus());
    }

    @Test
    public void testSetTemperature() {
        record.setTemperature(39.0);
        assertEquals(39.0, record.getTemperature(), 0.001);
    }

    @Test
    public void testSetWeight() {
        record.setWeight(72.5);
        assertEquals(72.5, record.getWeight(), 0.001);
    }

    @Test
    public void testSetRespiratoryStatus() {
        record.setRespiratoryStatus("yes");
        assertEquals("yes", record.getRespiratoryStatus());
    }

    @Test
    public void testDateInitialized() {
        assertNotNull("Date must be initialized in constructor", record.getDate());
    }
}
```



InteractiveChatTest

```
import ec.edu.espe.mothersApp.model.InteractiveChat;

import java.io.ByteArrayOutputStream;
import java.io.PrintStream;
import org.junit.Test;
import static org.junit.Assert.*;

/**
 *
 * @author Cristian
 */
public class InteractiveChatTest {
    private String captureMessage(String message) {
        ByteArrayOutputStream baos = new ByteArrayOutputStream();
        PrintStream original = System.out;
        System.setOut(new PrintStream(baos));
        try {
            InteractiveChat chat = new InteractiveChat();
            chat.sendMessage(message);
        } catch (Exception ignored) {}
        System.setOut(original);
        return baos.toString();
    }

    @Test
    public void testFeverKeyword() {
```



```

        String output = captureMessage("My baby has fever");
        assertTrue("Fever keyword should trigger temperature advice",
            output.toLowerCase().contains("temperature"));
    }

    @Test
    public void testFeedingKeyword() {
        String output = captureMessage("Feeding problem today");
        assertTrue("Feeding keyword should trigger feeding advice",
            output.toLowerCase().contains("feeding"));
    }

    @Test
    public void testCryingKeyword() {
        String output = captureMessage("Baby won't stop crying");
        assertTrue("Crying keyword should trigger crying advice",
            output.toLowerCase().contains("crying"));
    }

    @Test
    public void testSleepKeyword() {
        String output = captureMessage("Baby can't sleep");
        assertTrue("Sleep keyword should trigger sleep environment advice",
            output.toLowerCase().contains("quiet") ||
            output.toLowerCase().contains("room"));
    }

    @Test
    public void testUnknownKeyword() {
        String output = captureMessage("random unknown topic xyz");
        assertTrue("Unknown topic should return default community reply",
            output.contains("community") || output.contains("received"));
    }
}

```

